

CODE SPORT



This month's column is special, in honour of LFY's 9th anniversary. We feature tips on how to prepare for programming interviews.

This month, we celebrate the 9th anniversary of LFY. It's been a pleasure to be associated with the magazine. The CodeSport column has been primarily about helping newbie programmers and students learn the tricks of programming and problem solving, and enabling them to clear technical interviews. Therefore, for this special edition of CodeSport, we go back to the basics, featuring advice on how to prepare for and clear programming interviews.

What's so difficult about a programming interview?

I frequently get the following question from our readers: *"I have an interview coming up, which is primarily on programming. Can you give me pointers on how best to prepare for it? I have just 9 days left..."*

Well, there is no magic formula that can help you suddenly become an ace programmer in 9 days. Therefore, if you want to be an expert programmer, you need to read and practice diligently. You cannot cram all your preparation into 9 days and expect to succeed. Becoming an expert programmer and remaining one is a lifetime quest.

Make sure that your basics in computer science are clear and strong. Here is a simple example. Most interviewers, no matter for what position on the technical ladder they are interviewing the candidate, start off with the basics in computer science—such as algorithms, data structures, operating systems and the fundamentals of computer architecture. For instance, a candidate who cannot clearly enunciate when one would prefer to use counting sort rather than merge sort for sorting the given inputs is not someone you want to interview further. So get your basics right. For instance, in algorithms, you need to be familiar with Big O notation, the basics of searching—linear search

versus binary search, the basics of sorting—quick sort, merge sort, heap sort and counting sort, and the trade-offs involved in preferring one sorting algorithm over another. You need to know the different algorithmic approaches available, such as divide and conquer, greedy methods and dynamic programming. You are also expected to know about recursion, string-searching techniques, and the complexity classes of algorithms such as P, NP and NP Complete.

In data structures, the minimum expectation is that you should know the implementation of queues, stacks, linked lists, hash tables, heap, binary trees, binary search trees and some basics about balanced trees. In operating systems, you are expected to know about virtual memory, the basics of file-systems, interrupt handling, scheduling and synchronisation. In computer architecture, you should know about the trade-offs when choosing between a RISC and a CISC architecture, about Amdahl's law, pipe-lining, cache hierarchy, cache coherence, NUMA, TLB, etc. This is not an exhaustive list, but just a mere indication of the topics you should definitely be familiar with in order to convince an interviewer that your computer science basics are strong.

Expertise in one or two programming languages is important. If you are applying for a systems software position, typically C and C++ are the preferred languages. An important word of caution here: Many students claim that they have expertise in C++ when they have been using C++ as a glorified version of C, without really exploring its features with respect to templates, OOP concepts and STL. If you claim that you are an expert in C++, the interviewer will expect you to know all about how virtual functions are implemented, multiple inheritance, and template specialisation. It is not enough to know how to use the STL vector, queue or set/map. You need to know when you would choose one data structure over